

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	HARIHARAN G	Reg. No.:	192424156
Section / Batch:	B.TECH AIDS	Date:	

Code of Conduct

I HARIHARAN G(192424156)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: HARIHARAN G

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Porter Stemmer	class design, methods, encapsulation, and rule-based stemming	1

Annexure A – Coding Challenge

Section 1: Object-Oriented Implementation of a Porter Stemmer

You have recently joined an **NLP startup** called **LexiMind Technologies**. The company is developing a **search engine** for a digital library containing millions of research papers. Users expect the search engine to return relevant results even when different grammatical forms of a word are used.

For example, a search for **connect** should also retrieve documents containing **connected**, **connecting**, **connection**, and **connections**. To achieve this, the development team has decided to implement the **Porter Stemming Algorithm** from scratch.

Since the algorithm consists of multiple sequential rules (Step 1a, Step 1b, Step 1c, Step 2, Step 3, Step 4, Step 5a, and Step 5b), the project manager has divided the work among team members. Each developer is responsible for implementing and testing one specific rule. At the end of the project, all modules must be integrated into a single, fully functional stemmer.

Assessment Task

Phase 1: Individual Task (Assigned Rule)

Each student will be assigned **one Porter Stemmer rule**.

Your responsibilities are to:

1. Study and understand the assigned rule.
2. Explain the linguistic purpose of the rule.
3. Describe the conditions under which the rule should be applied.
4. Implement the rule in Python.
5. Prepare at least **10 test words** demonstrating:
 - Words that satisfy the rule.
 - Words that should remain unchanged.
6. Document the input, expected output, and actual output.
7. Explain any limitations or special cases encountered.

Phase 2: Team Integration

As a team, integrate all individual rule implementations into a complete Porter Stemmer.

The team must:

- Arrange the rules in the correct execution order.
- Resolve conflicts between rule implementations.
- Ensure that the output of one rule becomes the input to the next rule.
- Test the complete algorithm using at least **50 different English words**.
- Compare the team's results with the NLTK Porter Stemmer.
- Identify and explain any differences.

Phase 3: Reflection

Prepare a report addressing the following questions:

1. Why is the order of Porter Stemmer rules important?
2. What errors would occur if your assigned rule were skipped?
3. How did integrating multiple rules improve stemming accuracy?
4. Which rule was the most challenging to implement and why?
5. What improvements would you suggest for the Porter Stemming algorithm?

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach

Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Dr.T.Kumaragurubaran		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Title

Implementation of Porter Stemmer Step 3 Using Python

Aim

To implement **Step 3** of the Porter Stemming Algorithm in Python to remove or replace common suffixes and obtain the stem (root) form of a word.

Objective

- To understand the concept of stemming in Natural Language Processing (NLP).
 - To implement the suffix replacement rules of Porter Stemmer Step 3.
 - To convert derived words into their root forms for efficient text processing.
-

Theory

Stemming is the process of reducing a word to its base or root form by removing suffixes. It is widely used in **Natural Language Processing (NLP)**, search engines, text mining, and information retrieval. The **Porter Stemming Algorithm** consists of multiple steps. **Step 3** focuses on replacing or removing specific suffixes to simplify words while preserving their meaning.

Some common suffix transformations are:

Suffix Replacement Example

icate	ic	duplicate → duplic
ative	Remove	informative → inform
alize	al	formalize → formal
icity	ic	electricity → electric
ical	ic	logical → logic
ful	Remove	beautiful → beauti
ness	Remove	happiness → happi

Algorithm

1. Create a class named **PorterStep3**.
 2. Define a method called **stem()**.
 3. Check whether the input word ends with a particular suffix.
 4. If a suffix matches, remove or replace it according to Porter Step 3 rules.
 5. If no suffix matches, return the original word.
 6. Read a word from the user.
 7. Display both the original word and the stemmed word.
-

Program Explanation

Class Declaration

```
class PorterStep3:
```

- A class named **PorterStep3** is created.
- It contains the stemming logic for Step 3 of the Porter Algorithm.

Method Definition

def stem(self, word):

- Defines a method named **stem()**.
- self refers to the current object.
- word is the input string provided by the user.

Checking for "icate"

if word.endswith("icate"):

return word[:-5] + "ic"

- endswith() checks if the word ends with **"icate"**.
- word[:-5] removes the last five characters.
- "ic" is added.

Example

duplicate

↓

duplic + ic

↓

duplicic

(This program demonstrates the rule only; it does not implement all earlier Porter steps.)

Checking for "ative"

elif word.endswith("ative"):

return word[:-5]

- Removes the suffix **"ative"** completely.

Example

informative

↓

inform

Checking for "alize"

elif word.endswith("alize"):

return word[:-5] + "al"

- Replaces **"alize"** with **"al"**.

Example

formalize

↓

formal

Checking for "iciti"

elif word.endswith("iciti"):

return word[:-5] + "ic"

- Replaces **"iciti"** with **"ic"**.

Example

electricity

↓

electric

Checking for "ical"

```
elif word.endswith("ical"):
    return word[:-4] + "ic"
```

- Replaces **"ical"** with **"ic"**.

Example

logical

↓

logic

Checking for "ful"

```
elif word.endswith("ful"):
    return word[:-3]
```

- Removes the suffix **"ful"**.

Example

beautiful

↓

beauti

Checking for "ness"

```
elif word.endswith("ness"):
    return word[:-4]
```

- Removes the suffix **"ness"**.

Example

happiness

↓

happi

Default Return

```
return word
```

- If none of the suffixes match, the original word is returned unchanged.

Example

computer

↓

computer

Creating an Object

```
ps = PorterStep3()
```

- Creates an object named **ps** of the class **PorterStep3**.

Getting User Input

```
word = input("Enter a word: ")
```

- Reads a word entered by the user.

Displaying Output

```
print("Original Word :", word)
print("Stemmed Word :", ps.stem(word))
```

- Prints the original word.

- Calls the stem() method and prints the stemmed word.

Flow of Execution

Start



Create PorterStep3 Object



Read Input Word



Check "icate"?

↓ Yes → Replace with "ic"

↓ No

Check "ative"?

↓ Yes → Remove

↓ No

Check "alize"?

↓ Yes → Replace with "al"

↓ No

Check "iciti"?

↓ Yes → Replace with "ic"

↓ No

Check "ical"?

↓ Yes → Replace with "ic"

↓ No

Check "ful"?

↓ Yes → Remove

↓ No

Check "ness"?

↓ Yes → Remove

↓ No

Return Original Word



Display Result



End

Sample Input and Output

Example 1

Input

formalize

Output

Original Word : formalize

Stemmed Word : formal

Example 2

Input

logical

Output

Original Word : logical

Stemmed Word : logic

Example 3

Input

happiness

Output

Original Word : happiness

Stemmed Word : happi

Applications

- Search Engines
 - Information Retrieval
 - Text Mining
 - Sentiment Analysis
 - Chatbots
 - Machine Learning
 - Document Classification
-

Advantages

- Simple and easy to implement.
 - Reduces word variations.
 - Improves search efficiency.
 - Speeds up text processing.
 - Useful in many NLP applications.
-

Limitations

- Only implements **Step 3** of the Porter Stemmer.
 - Does not perform all stemming rules of the complete algorithm.
 - Some resulting stems may not be valid English words.
 - Works only with predefined suffixes.
-

Result

The Python program successfully implements **Step 3 of the Porter Stemming Algorithm** by checking predefined suffixes and removing or replacing them according to the Porter Stemmer rules. It correctly converts derived words into simplified stem forms suitable for Natural Language Processing applications.